
Rapport d'activité N°3

Semaine 49 à semaine 02

Rédigé par

CLOÉE HAREAU
STEVEN TRINH
ADRIEN BASCHUNG
WENJIA TANG

Table des matières

1	Gestion de projet	3
1.1	Généralités	3
1.2	Répartition du nombre d'heure.	3
1.2.1	Partie 3 : la TDS	3
1.3	Contexte / hypothèses de départ : Matrice SWOT	4
1.4	Déroulement du projet	4
2	Travail réalisé	6
2.1	Package tds	6
2.2	TdsCreator	6
2.3	Contrôles sémantiques effectués	6
3	Difficultés rencontrés	8
3.1	Récursivité mutuelle	8
3.2	Alias de type	8
3.3	Identifiants et LValues	8
3.4	Nom et conceptions des noeuds de l'AST	9
4	Annexes	10
4.1	Table des symboles et erreurs sémantiques	10
4.2	Comptes-rendus des réunions	12
4.2.1	Partie 1 : La grammaire LL(1)	12
4.2.2	Partie 2 : Création de AST	15
4.2.3	Partie 3 : Création de TDS	17

1 Gestion de projet

1.1 Généralités

Auteurs de ce cahier des charges / groupe d'expression du besoin

Nom / mail :	Qualité / rôle
Steven TRINH steven.trinh@telecomnancy.eu	Chef de Projet / Responsable de projet
Cloée HAREAU cloee.hareau@telecomnancy.eu	Projet / Membre de l'équipe
Adrien BASCHUNG adrien.baschung@telecomnancy.eu	Projet / Membre de l'équipe
Wenjia TANG Wenjia.Tang@telecomnancy.eu	Projet / Membre de l'équipe
Suzanne COLLIN suzanne.collin@univ-lorraine.fr	Commanditaire

Les réunions ont été planifiées en fonction des emplois du temps de chacun, favorisant les week-end et les vacances scolaires. Ainsi, la plupart des réunions se sont déroulées à distance sur la plateforme Discord.

Les différents documents du projet étaient partagés sur le Gitlab de TELECOM Nancy, très utilisé pour le partage de fichiers de code, les documents de la gestion de projet étaient eux partagés sur un Google Drive (et sont aussi disponibles sur le Gitlab).

1.2 Répartition du nombre d'heure.

1.2.1 Partie 3 : la TDS

N'ayant pas noté au fur et à mesure les heures de travaux le temps noté ci-dessous est approximatif.

Noms	Temps passé sur le rapport	Temps passé sur le code
Steven TRINH	1h	35h
Adrien BASCHUNG	1h	50h
Wenjia TANG	7h	31h
Cloée HAREAU	15h	25h

1.3 Contexte / hypothèses de départ : Matrice SWOT

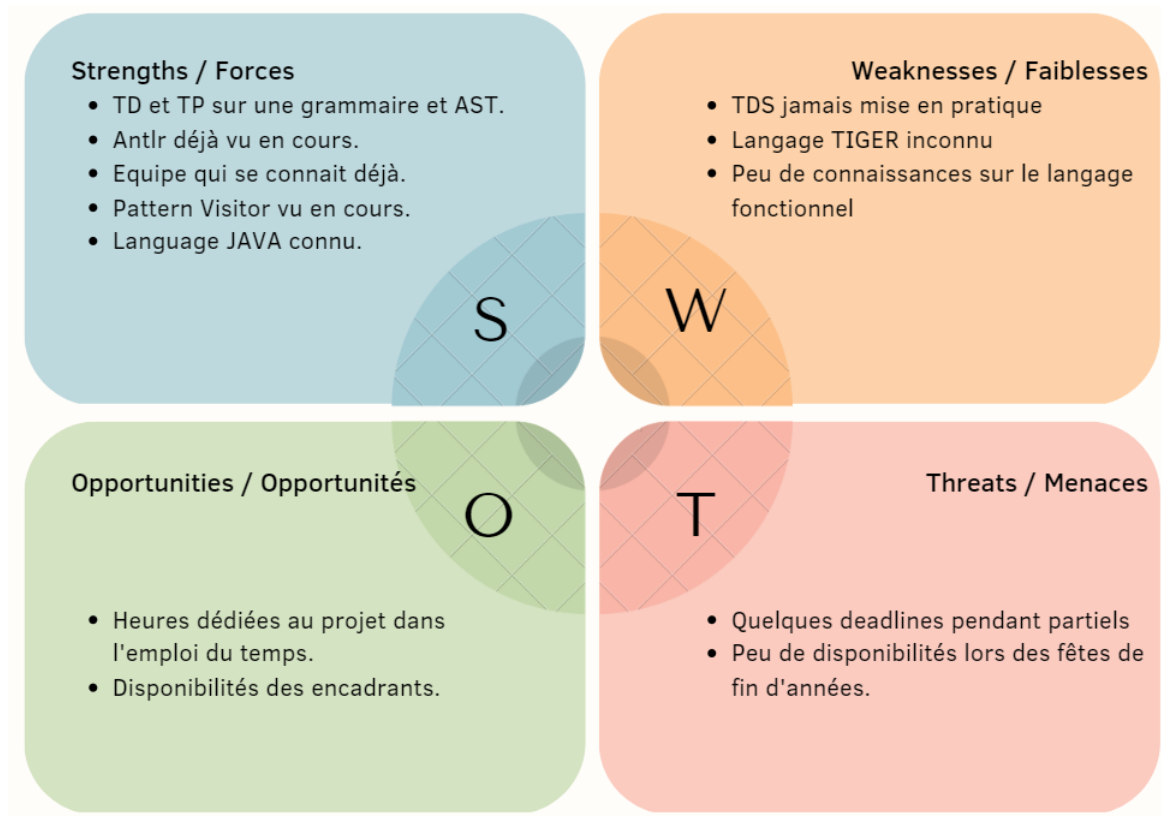


FIGURE 1.1 – Matrice SWOT

1.4 Déroulement du projet

Le projet prend place du mois de octobre 2022 jusqu'au mois de janvier 2023.

Il est composé de trois parties. La première consistait à construire une grammaire LL(1). Lors des deux semaines consacrées à la deuxième partie, nous avons écrit un code permettant de consulter les arbres syntaxiques des programmes écrits à l'aide du langage. La troisième partie se concentre sur la création de tables de symboles et des erreurs sémantiques à l'aide du contrôle sémantique.

Dans l'optique d'une meilleure organisation du projet, nous avons séparé les différentes parties de programmation du projet pour nous les répartir équitablement.

On retrouve ici le diagramme de Gantt pour la répartition des tâches.

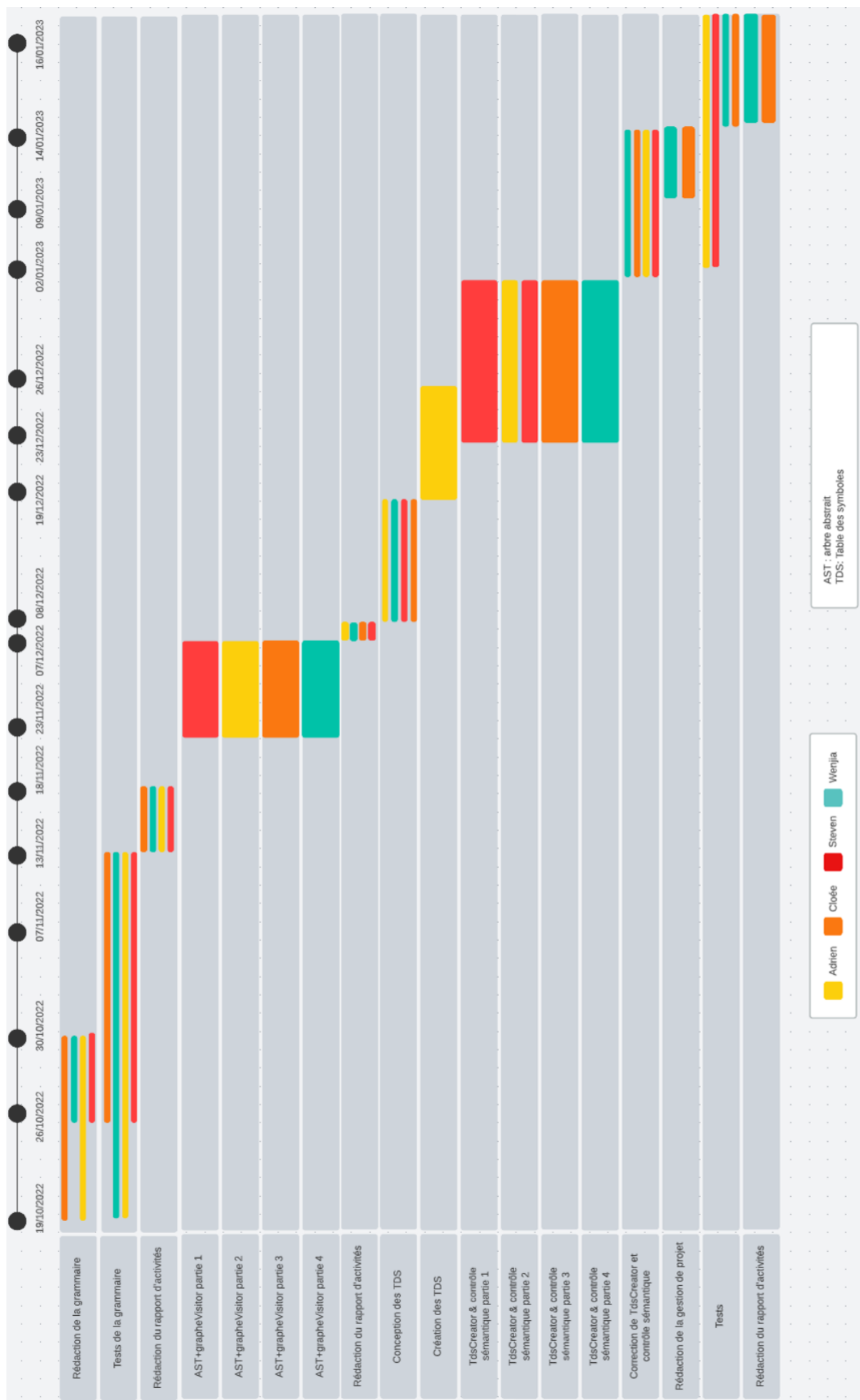


FIGURE 1.2 – Diagramme de Gantt

2 Travail réalisé

Suite à la création du générateur d'arbres abstraits, nous avons commencé à développer le générateur de tables de symboles et le contrôle sémantique. Pour cela, nous avons créé un package nommé `tds`, qui comprend l'ensemble des classes nécessaires. `TdsCreator`, qui implémente l'interface `AstVisitor`, est un élément du package `tds` utilisé pour créer les tables de symboles.

2.1 Package `tds`

Le package `tds` contient toutes les classes qui composent la table des symboles. Ainsi, la classe `Tds` représente la structure générale de la table des symboles. Les classes qui héritent de la classe `Entry` sont associées aux différents éléments de la table des symboles. Par exemple, la classe `ArrayEntry` est liée à l'élément *Array* de la table des symboles.

Ce package contient également des classes qui sont responsables d'une partie du contrôle sémantique. De cette manière, les classes qui implémentent l'interface `LaterVerif` se chargent de la vérification d'existence d'un type ou d'une fonction lors des déclarations (voir la section 3.1 sur la récursivité mutuelle).

2.2 `TdsCreator`

`TdsCreator` est le visiteur qui parcourt l'arbre abstrait pour générer la table des symboles et réaliser le contrôle sémantique. Ce visiteur ne retourne que des `String` qui vont contenir des identificateurs de variables, de fonctions, ou de types, ce qui nous a facilité l'implémentation du code. Aussi, il affiche sur la sortie standard les erreurs sémantiques détectées avec le numéro de ligne du bloc concerné. Excepté toutes les méthodes `visit`, cette classe contient une liste de tables des symboles, un identifiant de la table des symboles courante et des booléens nécessaires à notre programme. Elle contient également une liste des éléments à vérifier. Cette liste permet de réaliser, lors des déclarations, le contrôle sémantique par rapport à la récursivité mutuelle.

2.3 Contrôles sémantiques effectués

Au total, nous avons implémenté 15 contrôles sémantiques :

- l'existence des identificateurs appelés (type lors de déclarations (field, éléments, alias, paramètres, type de retour, variable), field/record/fonctions lors d'appels)
- la non réaffectation de la variable d'itération des boucles `for`
- la non réaffectation d'une fonction
- le bon type des membres lors des affectations
- le bon type des éléments/fields lors des appels de types
- le bon type des les opérandes pour les opérateurs (int, string ou seulement de même type)
- le bon type de retour pour l'expression de la condition et pour le corps des blocs `ifthen/ifthenelse/for/while`
- la nécessité pour les `break` d'être dans une boucle `for` ou `while`
- le bon nombre et type des paramètres donnés lors des appels de fonctions
- le bon nombre et les bons fields initialisés lors de l'appel d'un record

- la non redéfinition d'un type ou d'une variable/fonction dans la même table des symboles (de même portée)
- l'unicité des fields/paramètres dans un record/une fonction
- la nécessité que le type de l'identificateur de la cible lors d'appels d'éléments/initialisation d'un array soit un array, et lors d'appels de fields/initialisation d'un record soit un record
- la nécessité que l'identificateur lors d'un appel de fonction corresponde bien à une fonction
- le bon type de retour lors de la déclaration de fonctions

Les contrôles sémantiques sont effectués en prenant en compte la récursivité mutuelle, les alias et la séparation de l'espace de nommage des tables de symboles en deux parties : les variables/fonctions et les types.

3 Difficultés rencontrés

3.1 Récursivité mutuelle

Une des plus grandes difficultés à laquelle nous avons été confrontés était la gestion des fonctions et types récursifs ou mutuellement récursifs. Dans un premier temps, nous avons simplement prévu de parcourir l'arbre linéairement et d'effectuer les contrôles sémantiques et de créer les tables de symboles simultanément. Ainsi, nous pouvions vérifier si un symbole utilisé avait bien été défini au préalable. Cependant, nous avons dû modifier ce fonctionnement à cause de la récursivité mutuelle. Nous devons permettre à l'utilisateur d'utiliser une fonction ou un type avant qu'il ne soit déclaré, à condition d'être dans la même séquence de déclaration.

Pour résoudre ce problème nous avons donc modifier l'ordre de parcours de l'arbre, en particulier lors des déclarations. Tout d'abord nous avons ajoutés deux attributs booléens au visiteur `TdsCreator` de l'AST : `inTypeDecBloc` et `inFunctionDecBloc` permettant respectivement de savoir si l'arbre est en train de parcourir une séquence de déclarations de types ou de fonctions. De plus, dans chaque déclaration de type ou de fonction, certaines parties de l'AST susceptibles de contenir des symboles déclarés et accessibles par récursivité mutuelle sont stockés dans des classes `LaterVerif` qui effectueront les contrôles sémantiques plus tard, c'est-à-dire à la fin de la séquence de déclarations correspondante.

3.2 Alias de type

Ensuite, en finissant d'implémenter la création de l'entrée pour la déclaration d'un type dans la table des symboles, nous avons réalisé la nécessité de gérer les alias de types. En effet, à chaque contrôle sémantique sur les types, il nous faut pouvoir passer au-delà des éventuels alias de types afin de garantir la viabilité des contrôles.

Pour gérer cela, nous avons décidé de conserver seulement les types originels et non pas les alias pour chaque champs correspondant à un type dans les entrées de variables, de fonctions, ou de types (type des fields dans un record ou des éléments dans un array) de la table des symboles. Aussi, nous avons décidé d'ajouter une entrée spécifique pour les types alias qui comprend un pointeur vers l'entrée de son type originel.

Ainsi, les contrôles sémantiques sur les types n'avaient pas à être modifiés, et seul le getter d'entrées de types de la table des symboles avait à l'être pour pouvoir renvoyer l'entrée du type originel si un type alias était trouvé.

3.3 Identifiants et LValues

Vers la fin de cette partie, durant notre période de tests, nous avons remarqué un problème qui a nécessité d'importants changements sur le `TdsCreator`. Initialement, nous avons décidé que la visite du noeud `Idf` devrait renvoyer l'identifiant représenté par le noeud. Cependant un problème est apparu, la visite d'une expression en général renvoie en résultat le type de l'expression sous forme de string. Et il était alors compliqué voir impossible de gérer le cas particulier de l'`Idf`.

Nous avons donc choisi d'ajouter un attribut booléen `nameIdf` qui permettait d'indiquer au noeud `Idf` que l'on souhaite récupérer l'identifiant lui même dans certains cas particulier. Dans le cas contraire, on récupère le type de la variable correspondante si celle-ci est définie, si celle ci n'est pas définie, il renverra null. Cependant un nouveau problème est apparu avec l'utilisation de null, certaines parties

du code devenaient invalide à cause de variables null. Nous avons donc du relire l'ensemble du visiteur pour ajouter les vérifications qui s'imposent, à savoir vérifier dans certains cas que la variable ne vaut pas null.

3.4 Nom et conceptions des noeuds de l'AST

Pour terminer, nous avons également été confrontés à un problème de conception de l'AST.

Bien que nous ayons essayé de réfléchir à l'avance à la suite du projet et de préparer l'AST et la grammaire en conséquence, ce qui a tout de même considérablement facilité la réalisation de cette étape, nous avons tout de même commis quelques erreurs de conception :

- Le rôle des noeuds de l'Ast avait parfois été mal interprété et certains noeuds ont été mal nommés
- Certaines parties de l'arbre étaient mal formées, par exemple la déclaration de fonctions était séparée en deux noeuds, ce qui complexifiait sa gestion dans le **TdsCreator**.

Ainsi, il nous a fallu modifier le travail déjà réalisé, de la grammaire jusqu'au visiteur de la création de la table des symboles. Ces erreurs ne constituaient pas des problèmes difficiles à corriger mais ont constituées une perte de temps importante dans la réalisation de cette dernière étape.

4 Annexes

4.1 Table des symboles et erreurs sémantiques

1 - 1	VAR	Id : N	Type : int	
	VAR	Id : row	Type : intArray	
	VAR	Id : col	Type : intArray	
	VAR	Id : diag1	Type : intArray	
	VAR	Id : diag2	Type : intArray	
	PROC	Id : printboard	Param : 0	
	PROC	Id : try	Param : 1	c : int
	ARRAY	Id : intArray	Type : int	
2 - 2				
2 - 3	VAR	Id : c	Type : int	
3 - 4	VAR	Id : i	Type : int	
4 - 5	VAR	Id : j	Type : int	
3 - 6	VAR	Id : r	Type : int	

FIGURE 4.1 – TDS pour le programme du manuel

```

1 let /* The eight queens solver from Appel */
2   var N := 8
3   type intArray = array of int
4   var row := intArray [ N ] of 0
5   var col := intArray [ N ] of 0
6   var diag1 := intArray [ N+N-1 ] of 0
7   var diag2 := intArray [ N+N-1 ] of 0
8
9   function printboard() =
10     (for i := 0 to N-1 do
11       (for j := 0 to N-1 do
12         (print(if col[i]=j then (0) else (1))));
13       print(1));
14     print(1))
15
16   function try(c:int) =
17     if c=N then (printboard())
18     else (for r := 0 to N-1 do(
19       if row[r]=0 & diag1[r+c]=0 & diag2[r+7-c]=0 then
20         (row[r] := 1; diag1[r+c] := 1;
21          diag2[r+7-c] := 1; col[c] := r;
22          try(c+1);
23          row[r] := 0; diag1[r+c] := 0;
24          diag2[r+7-c] := 0)
25     ))
26
27 in try(0) end

```

```

Erreur ligne 3 : l'identificateur i ne doit pas être réaffecté
Erreur ligne 6 : les bornes ne sont pas des int
Erreur ligne 12 : l'identificateur i ne doit pas être réaffecté
Erreur ligne 14 : l'identificateur i ne doit pas être réaffecté
0 - 0

1 - 1
| VAR | Id : i | Type : int |
1 - 2
| VAR | Id : i | Type : int |
1 - 3
| VAR | Id : i | Type : int |
2 - 4
| VAR | Id : j | Type : int |

```

FIGURE 4.2 – Erreurs sémantiques et TDS pour des boucles for

```

1 (
2   for i := 1 to 5 do (
3     i := 0
4   );
5
6   for i := "a" to 5 do();
7
8   for i := 1 to 5 do (
9     let
10      var j := 0
11    in
12      i := 0
13    end;
14    i :=0
15  )
16 )

```

4.2 Comptes-rendus des réunions

4.2.1 Partie 1 : La grammaire LL(1)

Réunion du 09/10/2022
ÉCRIT PAR CLOÉE HAREAU

Motif / type de réunion : Réunion Brainstorming	Lieu : Réunion en présentiel (Telecom)
Présents : Adrien BASCHUNG, Wenjia TANG, Steven TRINH, Cloée HAREAU	Date : 19/10/2022 - 15h Durée : 1h

Ordre du jour

- Discuter du sujet, partager les idées sur l'écriture de la grammaire.
- Répartition du travail à réaliser.
- Commencer la rédaction début de grammaire
- Création du dépôt GIT

Décisions

- La grammaire proposée n'est pas LL(1) Il faut donc la modifier.

Todo List :

Description	Responsable	Délai	Livrable	Validé par
Continuer la grammaire	Adrien BASCHUNG Cloée HAREAU	26/10/2022 15h00	Grammaire	Toute l'équipe
Écriture de tests	Steven TRINH Wenjia TANG	26/10/2022 15h00	Codes TIGER	Toute l'équipe

Prochaine réunion : 26/10/2022

Réunion du 26/10/2022
ÉCRIT PAR STEVEN TRINH

Motif / type de réunion : Réunion d'avancement	Lieu : Réunion en distanciel (Discord)
Présents : Adrien BASCHUNG, Wenjia TANG, Steven TRINH, Cloée HAREAU	Date : 26/10/2022 - 15h Durée : 1h

Ordre du jour

- Continuer la grammaire
- Vérification du travail déjà effectué. (Enlever ambiguïté restante, les récursivités gauches, LL(1))

Décisions

- Il y a des soucis avec les opérateurs binaires donc il faut séparer les opérateurs du reste.
- Il reste des problèmes avec la compréhension de la grammaire donc poser des questions à Madame Collin ou Monsieur Da Silva :
 - "if a=0 then 3 else 4 + 2 $\rightarrow$ (if a=0 then 3 else 4)+2 ou if a=0 then 3 else (4+2)"
 - Les fonctions de la librairie sont-elles à coder ou à écrire dans la grammaire ?

Todo List :

Description	Responsable	Délai	Livrable	Validé par
Continuer la grammaire	Tous	30/10/2022 15h00	Grammaire	Toute l'équipe
Écrire des tests	Tous	30/10/2022 15h00	Codes TIGER	Toute l'équipe

Prochaine réunion : 30/10/2022

Réunion du 30/10/2022
ÉCRIT PAR WENJIA TANG

Motif / type de réunion : Réunion d'avancement	Lieu : Réunion en distanciel (Discord)
Présents : Adrien BASCHUNG, Wenjia TANG, Steven TRINH, Cloée HAREAU	Date : 30/10/2022 - 15h Durée : 1h

Ordre du jour

- Continuer la grammaire, discuter et partager des idées.
 - type_id [expr] of expr
 - type_id field_list
 - gen_id
 - opérateurs binaires
- Vérification du travail déjà effectué. (Enlever ambiguïté restante, les récursivités gauches, LL(1))

Todo List :

Description	Responsable	Délai	Livrable	Validé par
Écrire des tests	Tous	07/11/2022 15h00	Codes TIGER	Toute l'équipe
Vérifier la grammaire avec tests	Tous	07/11/2022 15h00	Grammaire	Toute l'équipe

Prochaine réunion : 13/11/2022

Réunion du 13/11/2022
ÉCRIT PAR ADRIEN BASCHUNG

Motif / type de réunion : Réunion d'avancement	Lieu : Réunion en distanciel (Discord)
Présents : Adrien BASCHUNG, Wenjia TANG, Steven TRINH, Cloée HAREAU	Date : 13/11/2022 - 15h Durée : 1h

Ordre du jour

- Terminer les dernières modifications de la grammaire

Décisions

- La grammaire est bonne.

Todo List :

Description	Responsable	Délai	Livrable	Validé par
Écrire rapport d'activités travail réalisé	Cloée HAREAU Wenjia TANG	18/11/2022 15h00	Fichier LATEX	Toute l'équipe
Ecrire rapport d'activités difficultés rencontrées	Steven TRINH Adrien BASCHUNG	18/11/2022 15h00	Fichier LATEX	Toute l'équipe

Prochaine réunion : 24/11/2022

4.2.2 Partie 2 : Création de AST

Réunion du 23/11/2022
ÉCRIT PAR CLOÉE HAREAU

Motif / type de réunion : Réunion type Brainstorming	Lieu : Réunion en présentiel (Telecom)
Présents : Adrien BASCHUNG, Wenjia TANG, Steven TRINH, Cloée HAREAU	Date : 23/11/2022 - 14h Durée : 1h

Ordre du jour

- Discuter des techniques vues en cours/TD
- Couper la grammaire en 4 parties
- Distribution de la charge de travail

Décisions

- Partie 1 : Steven, partie 2 : Adrien, partie 3 : Cloée, partie 4 : Wenjia.

Todo List :

Description	Responsable	Délai	Livrable	Validé par
Écrire le code pour sa partie	Tous	06/12/2022 19h00	Fichier Code JAVA	Toute l'équipe

Prochaine réunion : 30/11/2022

Réunion du 30/11/2022
ÉCRIT PAR WENJIA TANG

Motif / type de réunion : Réunion d'avancement	Lieu : Réunion en distanciel (Discord)
Présents : Adrien BASCHUNG, Wenjia TANG, Steven TRINH, Cloée HAREAU	Date : 30/11/2022 - 15h30 Durée : 1h

Ordre du jour

- Vérifier l'avancement du travail

Décisions

- L'avancement est bon, pour ceux qui ont déjà terminé, aider les autres ou commencer les tests.

Todo List :

Description	Responsable	Délai	Livrable	Validé par
Terminer le code	Tous	06/12/2022 19h00	Fichiers JAVA	Toute l'équipe

Prochaine réunion : 06/12/2022

Réunion du 06/12/2022
ÉCRIT PAR ADRIEN BASCHUNG

Motif / type de réunion : Réunion d'avancement	Lieu : Réunion en distanciel (Discord)
Présents : Adrien BASCHUNG, Wenjia TANG, Steven TRINH, Cloée HAREAU	Date : 06/12/2022 - 19h Durée : 1h30

Ordre du jour

- Terminer les dernières modifications du code.

Décisions

- La partie AST est terminée.

Todo List :

Description	Responsable	Délai	Livrable	Validé par
Écrire rapport d'activités travail réalisé	Cloée HAREAU Wenjia TANG	07/12/2022 14h00	Fichier LATEX	Toute l'équipe
Écrire rapport d'activités difficultés rencontrées	Steven TRINH Adrien BASCHUNG	07/12/2022 14h00	Fichier LATEX	Toute l'équipe

Prochaine réunion : 09/12/2022

4.2.3 Partie 3 : Création de TDS

Réunion du 09/12/2022 ÉCRIT PAR STEVEN TRINH

Motif / type de réunion : Réunion type Brainstorming	Lieu : Réunion en présentiel (Telecom)
Présents : Adrien BASCHUNG, Wenjia TANG, Steven TRINH, Cloée HAREAU	Date : 09/12/2022 - 10h Durée : 30 min

Ordre du jour

- Discuter de la structure et des données contenues dans la TDS.
- Faire un diagramme de classe pour la structure de la TDS.

Décisions

- Acceptation de la structure proposée par le diagramme.

Todo List :

Description	Responsable	Délai	Livrable	Validé par
Commencer le code du modèle pour ceux qui ont le temps	Tous	19/12/2022 19h00	Fichiers JAVA	Toute l'équipe

Prochaine réunion : 19/12/2022

Réunion du 19/12/2022 ÉCRIT PAR CLOÉE HAREAU

Motif / type de réunion : Réunion d'avancement	Lieu : Réunion en distanciel (Discord)
Présents : Adrien BASCHUNG, Wenjia TANG, Steven TRINH, Cloée HAREAU	Date : 19/12/2022 - 19h Durée : 45 min

Ordre du jour

- Vérifier le travail déjà fait.
- Terminer le code du modèle.
- Attribution de parties de la grammaire pour coder la création de la TDS.

Décisions

- Partie 1 : Steven, partie 2 : Adrien, partie 3 : Cloée, partie 4 : Wenjia.

Todo List :

Description	Responsable	Délai	Livrable	Validé par
Commencer le code pour ceux qui ont le temps	Tous	28/12/2022 22h00	Fichiers JAVA	Toute l'équipe

Prochaine réunion : 28/12/2022

Réunion du 28/12/2022
ÉCRIT PAR ADRIEN BASCHUNG

Motif / type de réunion : Réunion d'avancement	Lieu : Réunion en distanciel (Discord)
Présents : Adrien BASCHUNG, Wenjia TANG, Steven TRINH, Cloée HAREAU	Date : 28/12/2022 - 22h Durée : 30 min

Ordre du jour

- Montrer l'avancement du travail

Todo List :

Description	Responsable	Délai	Livrable	Validé par
Continuer les anciennes tâches : sa partie sur TDSCreator	Tous	02/01/2023 10h00	Fichier JAVA	Toute l'équipe

Prochaine réunion : 02/01/2023

Réunion du 02/01/2023
ÉCRIT PAR WENJIA TANG

Motif / type de réunion : Réunion d'avancement	Lieu : Réunion en distanciel (Discord)
Présents : Adrien BASCHUNG, Wenjia TANG, Steven TRINH, Cloée HAREAU	Date : 02/01/2023 - 10h30 Durée : 1h

Ordre du jour

- Présenter le travail déjà effectué.
- Discuter des difficultés rencontrées.

Décisions

- Modifications des classes du modèle.
- Modifications de structure dans les déclarations.

Todo List :

Description	Responsable	Délai	Livrable	Validé par
Continuer le travail attribué	Cloée HAREAU Wenjia TANG	08/01/2023 10h00	Fichier JAVA	Toute l'équipe
Aider sur la partie des déclarations	Steven TRINH Adrien BASCHUNG	08/01/2023 10h00	Fichier JAVA	Toute l'équipe

Prochaine réunion : 08/01/2023

Réunion du 08/01/2023
ÉCRIT PAR CLOÉE HAREAU

Motif / type de réunion : Réunion d'avancement	Lieu : Réunion en distanciel (Discord)
Présents : Adrien BASCHUNG, Wenjia TANG, Steven TRINH, Cloée HAREAU	Date : 08/01/2023 - 16h Durée : 1h

Ordre du jour

- Revoir les problèmes sur les nouvelles déclarations

Todo List :

Description	Responsable	Délai	Livrable	Validé par
Commencer la rédaction du compte rendu	Cloée HAREAU Wenjia TANG	14/01/2023 14h00	Fichier LATEX	Toute l'équipe
Corrections de bugs et tests	Tous	14/01/2023 14h00	Fichiers JAVA	Toute l'équipe

Prochaine réunion : 15/01/2023

Réunion du 15/01/2023
ÉCRIT PAR STEVEN TRINH

Motif / type de réunion : Réunion d'avancement	Lieu : Réunion en distanciel (Discord)
Présents : Adrien BASCHUNG, Wenjia TANG, Steven TRINH, Cloée HAREAU	Date : 15/01/2023 - 14h Durée : 1h30

Ordre du jour

- Terminer les fichiers JAVA.

Décisions

- La 3 partie TDS est terminée.

Todo List :

Description	Responsable	Délai	Livrable	Validé par
Terminer la rédaction du compte rendu.	Tous	15/01/2023 20h00	Fichier LATEX	Toute l'équipe